END-OF-SEMESTER DESIGN PROJECT • WEEKS 11–16 • TEAMS OF 3

The DSD-8 Processor

Design and Verification Specification

An 8-bit Harvard microprocessor in Verilog

Digital System Design

Senior Undergraduate Elective • Document version 1.0

What this document is

This is the *complete and binding* definition of the processor you are to build. Everything in Sections 2 to 6 is **frozen**: the instruction set, the encodings, the register model, the memory map, the module boundaries and the port names are identical for every team. Your design will be assessed by running the same programs on it that run on everybody else's.

Section 7 defines how your work is verified, Section 8 what you must hand in and when. Read Sections 2 and 3 before you write a single line of Verilog.

Why the architecture is fixed

Because it makes the difficult part of the project the *right* difficult part. You are not being asked to invent an instruction set; you are being asked to build a machine that correctly implements one, and to *prove* that it does. A frozen architecture means a shared assembler, a shared reference simulator, and a conformance suite that tells you objectively how far you have got — so that when your processor misbehaves you can find out *which instruction*, on *which cycle*, went wrong, instead of guessing.

Supplied to you with this document: the DSD-8 assembler (`dsd8asm.py`), the golden reference simulator (`dsd8sim.py`), the conformance test suite (40 programs with expected traces), and the testbench harness that runs a program and compares your trace against the reference.

Contents

1 Scope and Objectives **4**
1.1 What you are building . . . 4
1.2 Learning outcomes . . . 4
1.3 What is frozen and what is yours . . . 4
1.4 Notation used in this document . . . 5

2 Programmer's Model **6**
2.1 Overview . . . 6
2.2 Registers . . . 6
2.3 Condition flags . . . 6
2.4 Memory . . . 8
2.5 Reset state . . . 8
2.6 Halting . . . 8

3 Instruction Set Architecture **9**
3.1 Instruction formats . . . 9
3.2 Opcode map . . . 10
3.3 Branch conditions . . . 10
3.4 Instruction reference . . . 11
3.5 Worked encoding examples . . . 12
3.6 Assembly language . . . 13

4 System Architecture **14**
4.1 Block diagram . . . 14
4.2 Required module hierarchy . . . 14
4.3 Microarchitecture: your choice, with one constraint . . . 15

5 Module Interfaces **16**
5.1 `dsd8_top` — system top level . . . 16
5.2 `dsd8_cpu` — the processor core . . . 16
5.3 `prog_counter` . . . 16
5.4 `instr_memory` . . . 16
5.5 `instr_decoder` — purely combinational . . . 16
5.6 `register_file` . . . 17
5.7 `alu` . . . 17
5.8 `flags_reg` . . . 17
5.9 `data_memory` . . . 17
5.10 `control_unit` . . . 17

6 Control Unit and Datapath Operation **19**
6.1 Recommended state machine . . . 19
6.2 Control signal table . . . 19
6.3 Register operand routing . . . 20

7 Input / Output Subsystem **21**
7.1 Bus protocol . . . 21
7.2 Standard port map . . . 21
7.3 Polling example . . . 22

8 Verification **23**
8.1 The strategy, in one paragraph . . . 23
8.2 Execution trace format . . . 23
8.3 Harness . . . 24
8.4 Conformance test suite . . . 24
8.5 Your own verification — required beyond conformance . . . 26

9 Deliverables, Milestones and Assessment **27**
9.1 Milestones . . . 27
9.2 Mark allocation . . . 27

9.3 Final deliverables 27
9.4 Sample defence questions 28

A Encoding Quick Reference **29**

B Worked Example Programs **30**
B.1 Multiply by repeated addition 30
B.2 Population count 30
B.3 Bubble sort in data memory 30
B.4 Polled UART echo 31

C Common Pitfalls **32**

1 Scope and Objectives

1.1 What you are building

A working **8-bit microprocessor in Verilog**, complete with instruction memory, data memory, a register file, an ALU, a control unit and a memory-mapped input/output port interface. It must execute programs written in the DSD-8 assembly language, and it must produce results that match the reference simulator exactly, instruction for instruction.

The machine is deliberately small. It has eight registers, sixteen opcodes, 512 words of program memory and 256 bytes of data memory. It is not a fast machine and it is not a clever one. It is a machine you can completely understand, completely verify, and completely explain — which is the point.

1.2 Learning outcomes

On completing this project you will be able to:

1. Translate a written architectural specification into a working RTL implementation, without the specification being ambiguous enough to hide behind.
2. Partition a system into modules with explicit interface contracts, and integrate modules that were developed separately.
3. Design a control unit as a finite state machine driving a datapath, and produce the control-signal table that defines it.
4. Verify a sequential system by *trace comparison* against a golden model — the technique that transfers directly to industrial processor verification.
5. Diagnose a fault in a system with thousands of internal states, from evidence rather than by guesswork.
6. Defend every design decision you made, in person, without notes.

1.3 What is frozen and what is yours

Frozen — identical for every team	Yours to decide
Instruction set, encodings and semantics (§3)	The microarchitecture: single-cycle, multi-cycle or pipelined
Register model, memory sizes, reset state (§2)	How the control unit is structured and encoded
Module names and port names (§5)	Everything inside each module
The I/O bus protocol and port map (§6)	Whether the ALU is one block or several
The execution trace format (§7.2)	Coding style, provided it passes the lint rules
The conformance test suite (§7.4)	Your own additional tests, of which you will need many

> **The one rule that overrides everything else**
>
> If your processor does not produce the specified trace for the supplied test programs, it does not matter how elegant the inside is. Conformance is assessed first and objectively. Build the plainest machine that works, get it conforming, and only then improve it.

1.4 Notation used in this document

Notation	Meaning
`0x1F`	hexadecimal constant. `0b1010` is binary; a bare number is decimal.
`R[n]`	general-purpose register n, where $0 \le n \le 7$
`DMEM[a]`	the byte of data memory at address `a`
`IMEM[a]`	the 16-bit instruction word at address `a`
`IR[15:12]`	bits 15 down to 12 of the instruction word, bit 15 being the most significant
$\leftarrow$	assignment, taking effect at the end of the instruction
R*n* in the margin	a numbered, testable *requirement*. Every one of these is checked, either by the conformance suite or at the design review.

2 Programmer's Model

This section describes the machine as software sees it. Everything here is architecturally visible: it is what a program can observe, and therefore it is what your implementation must reproduce exactly. How you build it internally is not constrained.

2.1 Overview

Property	Value
Data width	8 bits
Instruction width	16 bits, fixed — every instruction is exactly one word
General-purpose registers	8, named `R0` to `R7`, 8 bits each, all equally usable
Program counter	9 bits, addressing 512 instruction words
Instruction memory	512 × 16 bits, read-only to the program (Harvard architecture)
Data memory	256 × 8 bits, addressed by the full range of an 8-bit register
Input/output	256 ports, in a separate address space, reached only by `IN` and `OUT`
Condition flags	four: Z, N, C, V
Stack	none. There are no subroutine calls in DSD-8.

> **Why Harvard, and why a separate I/O space?**
> Separating instruction and data memory means the fetch of the next instruction never competes with a load or store, which keeps the control unit simple enough to reason about. Putting I/O in its own address space means adding a peripheral never disturbs the data memory map — the two subsystems cannot interfere with one another. Both choices trade a little generality for a great deal of clarity, which is the correct trade for a first processor.

2.2 Registers

R1 The processor shall provide eight general-purpose 8-bit registers, `R0` through `R7`. None is hardwired to a constant; all eight may be freely read and written by any instruction that takes a register operand.

R2 Register `R0` is *not* special. Code that assumes it reads as zero is incorrect.

2.3 Condition flags

Four single-bit flags record properties of the most recent flag-setting result.

Flag	Name	Set to 1 when
`Z`	Zero	the 8-bit result is `0x00`
`N`	Negative	bit 7 of the result is 1 (the sign bit, if the value is read as two's complement)
`C`	Carry	the 9th bit of the internal addition is 1 — see below
`V`	Overflow	the result is wrong when the operands are read as *signed* values

2.3.1 The carry flag, stated exactly

This is the single most common source of disagreement between an implementation and the reference, so it is defined here without ambiguity.

Both `ADD` and `SUB` are performed by the *same* 9-bit addition inside the ALU:

Operation	Internal computation
ADD	`{carry, result} = a + b + 0`
SUB, CMP	`{carry, result} = a + (~b) + 1`

`C` is that carry out, taken directly. The consequence, which you should commit to memory:

After `CMP Ra, Rb`	Meaning
`C = 1`	`Ra` $\geq$ `Rb`, treating both as **unsigned**. No borrow occurred.
`C = 0`	`Ra` $<$ `Rb` unsigned. A borrow occurred.
`Z = 1`	`Ra` $=$ `Rb`

> **The trap**
> Some architectures define the subtract carry the other way round, so that `C = 1` means "a borrow happened". DSD-8 does **not**. If your `BCS` branches the wrong way after a `CMP`, this convention is the first thing to check. Test group T3 exists specifically to catch it.

2.3.2 Overflow

`V` indicates that the signed interpretation of the result is wrong. It is computed from the sign bits of the operands and of the result:

Operation	Overflow condition
ADD	`V = (a[7] == b[7]) && (result[7] != a[7])`
SUB, CMP	`V = (a[7] != b[7]) && (result[7] != a[7])`

In words: adding two numbers of the *same* sign and getting the other sign is an overflow; subtracting a number of the *opposite* sign and getting the wrong sign is an overflow. Adding numbers of opposite signs can never overflow.

2.3.3 Which instructions change the flags

R3 Exactly eight instructions modify the flags: `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SHL`, `SHR` and `CMP`. Every other instruction — including `LDI`, `LD`, `ST`, `IN`, `OUT`, `JMP`, the conditional branches, `NOP` and `HLT` — leaves all four flags completely unchanged.

Instruction	Z	N	C	V
ADD, SUB	result	result	carry out	signed overflow
CMP	result	result	carry out	signed overflow
AND, OR, XOR	result	result	cleared to 0	cleared to 0
SHL	result	result	bit 7 shifted out	cleared to 0
SHR	result	result	bit 0 shifted out	cleared to 0
all others			*unchanged*	

> **A deliberate design decision, worth understanding**
>
> `LD` does not set flags. That means you can load a value and branch on a comparison made *before* the load, which occasionally saves an instruction — but it also means `LD` followed by `BEQ` tests the *old* flags, not the value just loaded. Real architectures differ on this, and the ones that differ from your expectation cause real bugs. Note it now.

2.4 Memory

R4 Instruction memory holds 512 words of 16 bits, at addresses `0x000` to `0x1FF`. It is initialised from a hexadecimal image file at the start of simulation and is not writable by any instruction.

R5 Data memory holds 256 bytes at addresses `0x00` to `0xFF`. Because addresses come from an 8-bit register, the whole of data memory is reachable and no address can ever be out of range.

2.5 Reset state

The reset state must be exactly reproducible, because the reference simulator starts from it and your trace is compared against the reference from the very first instruction.

R6 On the release of reset the processor shall be in this state, with no exceptions: `PC = 0x000`; `R[0..7] = 0x00`; `Z = N = C = V = 0`; every byte of `DMEM = 0x00`; `halted = 0`. Execution begins at `IMEM[0]`.

R7 Reset shall be **synchronous** and **active low** (`rst_n`), asserted for at least two clock cycles before execution begins.

> **Do not skip the data-memory clear**
>
> It is tempting to leave data memory uninitialised, since a well-written program writes before it reads. The conformance suite does not assume that. Test T09 reads uninitialised locations deliberately, and expects `0x00`. In a simulation, an uninitialised memory reads as `x` and will propagate through your whole datapath.

2.6 Halting

R8 The `HLT` instruction shall stop execution: the program counter stops advancing, no further instructions are retired, and the output `halted` is asserted and remains asserted until reset.

R9 A processor that has halted shall not change any register, flag or memory location.

3 Instruction Set Architecture

Every instruction is exactly **16 bits**. The opcode always occupies `IR[15:12]`, so decoding always begins the same way. There are sixteen opcodes and every one of them is defined — there is no such thing as an illegal opcode in DSD-8.

3.1 Instruction formats

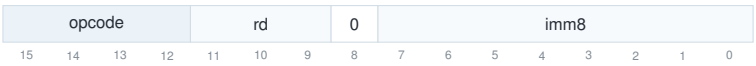
Five formats share the same 4-bit opcode field. Bit 15 is on the left.

Format R — register to register



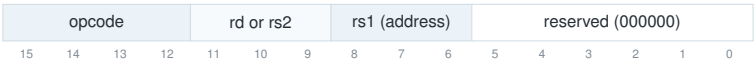
Used by `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SHL`, `SHR`, `CMP`. `SHL` and `SHR` ignore `rs2`; `CMP` ignores `rd`.

Format I — immediate



Used by `LDI` only. The 8-bit immediate occupies the low byte of the instruction word.

Format M — memory



Used by `LD` and `ST`. `rs1` always supplies the address.

Format J — jump and branch



Used by `JMP` (where the `cond` field is `000` and ignored) and by the conditional branches. The target is an **absolute** 9-bit instruction address.

Format P — port input/output



Used by `IN` and `OUT`. `dir` = 0 selects `IN`, `dir` = 1 selects `OUT`.

R10 Bits marked *reserved* shall be written as zero by the assembler. The behaviour of the processor when a reserved bit is non-zero is undefined; your implementation may do anything, and the conformance suite will never present such an instruction.

3.2 Opcode map

Opcode	Mnemonic	Format	One-line description
`0x0`	`NOP`	—	do nothing
`0x1`	`LDI rd, imm8`	I	load an 8-bit constant into a register
`0x2`	`LD rd, [rs1]`	M	load a byte from data memory
`0x3`	`ST [rs1], rs2`	M	store a byte to data memory
`0x4`	`ADD rd, rs1, rs2`	R	add, setting all four flags
`0x5`	`SUB rd, rs1, rs2`	R	subtract, setting all four flags
`0x6`	`AND rd, rs1, rs2`	R	bitwise AND
`0x7`	`OR rd, rs1, rs2`	R	bitwise OR
`0x8`	`XOR rd, rs1, rs2`	R	bitwise exclusive OR
`0x9`	`SHL rd, rs1`	R	shift left one place, MSB into carry
`0xA`	`SHR rd, rs1`	R	shift right one place (logical), LSB into carry
`0xB`	`CMP rs1, rs2`	R	compare: set flags, discard the result
`0xC`	`JMP addr9`	J	unconditional jump
`0xD`	`B\emph {cc} addr9`	J	branch if condition *cc* holds
`0xE`	`IN rd, port` / `OUT port, rs`	P	read from / write to an I/O port
`0xF`	`HLT`	—	stop execution

3.3 Branch conditions

The `cond` field of a Format J instruction with opcode `0xD` selects which flag is tested.

cond	Mnemonic	Branch taken if	Typical use, after `CMP Ra, Rb`
`000`	`BEQ`	`Z = 1`	`Ra` equals `Rb`
`001`	`BNE`	`Z = 0`	`Ra` differs from `Rb`
`010`	`BCS`	`C = 1`	`Ra` $\geq$ `Rb`, unsigned
`011`	`BCC`	`C = 0`	`Ra` $<$ `Rb`, unsigned
`100`	`BMI`	`N = 1`	result was negative
`101`	`BPL`	`N = 0`	result was positive or zero
`110`	`BVS`	`V = 1`	signed overflow occurred
`111`	`BVC`	`V = 0`	no signed overflow

There is no signed less-than branch — and that is on purpose

To compare signed values you must combine `N` and `V` yourself: `Ra < Rb` signed is true when $N \neq V$. Working out how to do that with the branches you have is a genuinely instructive exercise, and it is why `BVS` and `BVC` exist. Test group T4 requires it.

3.4 Instruction reference

In the tables below, `PC` is the address of the instruction being executed. Unless stated otherwise the next instruction executed is at `PC + 1`, and this wraps within the 9-bit address range.

3.4.1 Data movement

Instruction	Operation	Notes
`NOP`	none	Encoded as `0x0000`. Flags unchanged.
`LDI rd, imm8`	`R[rd] <- imm8`	The only way to get a constant into a register. Flags unchanged.
`LD rd, [rs1]`	`R[rd] <- DMEM[R[rs1]]`	The address is the *contents* of `rs1`, not the register number. Flags unchanged.
`ST [rs1], rs2`	`DMEM[R[rs1]] <- R[rs2]`	Note the operand order: address register first. No register is written. Flags unchanged.

3.4.2 Arithmetic and logic

Instruction	Operation	Flags
`ADD rd, rs1, rs2`	`R[rd] <- R[rs1] + R[rs2]`	Z, N from result; C = carry out; V = signed overflow
`SUB rd, rs1, rs2`	`R[rd] <- R[rs1] - R[rs2]`	as `ADD`; C = 1 means no borrow
`AND rd, rs1, rs2`	`R[rd] <- R[rs1] & R[rs2]`	Z, N from result; C = 0; V = 0
`OR rd, rs1, rs2`	`R[rd] <- R[rs1] | R[rs2]`	as `AND`
`XOR rd, rs1, rs2`	`R[rd] <- R[rs1] ^ R[rs2]`	as `AND`
`SHL rd, rs1`	`R[rd] <- R[rs1] « 1`	Z, N from result; C = old bit 7; V = 0. Zero shifted into bit 0.
`SHR rd, rs1`	`R[rd] <- R[rs1] » 1`	Logical, **not** arithmetic: zero shifted into bit 7. C = old bit 0; V = 0.
`CMP rs1, rs2`	flags from `R[rs1] - R[rs2]`	Identical to `SUB` except that **no register is written**.

`SHR` is logical, not arithmetic

`SHR` always shifts a zero into bit 7, even when the value is negative in two's complement. There is no arithmetic right shift in DSD-8. If you need one, you must construct it — which test T22 asks you to think about.

3.4.3 Control flow

Instruction	Operation	Notes
`JMP addr9`	`PC <- addr9`	Absolute address in the range `0x000`–`0x1FF`. Flags unchanged.
`B\emph {cc} addr9`	if condition holds, `PC <- addr9`; otherwise `PC <- PC + 1`	The flags are *tested*, never modified.
`HLT`	stop; assert `halted`	`PC` holds its own address. Nothing further changes.

3.4.4 Input and output

Instruction	Operation	Notes
`IN rd, port`	`R[rd] <- io_rdata(port)`	Reads the I/O space, never data memory. Flags unchanged. An unmapped port returns `0x00`.
`OUT port, rs`	`io_wdata(port) <- R[rs]`	No register is written. Flags unchanged. A write to an unmapped port is discarded.

R11 Both `IN` and `OUT` shall honour the `io_ready` handshake described in §7. If the addressed peripheral holds `io_ready` low, the processor shall stall — holding the program counter, the registers, the flags and the bus request steady — until `io_ready` is asserted, and only then complete the instruction.

3.5 Worked encoding examples

Read these carefully. Most encoding bugs are caught by hand-checking three or four instructions before running anything.

Assembly	Machine code	Field breakdown
`LDI R2, 13`	`0x140D`	opcode `0001`, rd `010`, bit8 `0`, imm `00001101`
`ADD R3, R3, R1`	`0x46C8`	opcode `0100`, rd `011`, rs1 `011`, rs2 `001`, rsvd `000`
`LD R5, [R3]`	`0x2AC0`	opcode `0010`, rd `101`, rs1 `011`, rsvd `000000`
`ST [R3], R0`	`0x30C0`	opcode `0011`, rs2 `000`, rs1 `011`, rsvd `000000`
`CMP R2, R5`	`0xB0A8`	opcode `1011`, rd `000` (unused), rs1 `010`, rs2 `101`
`JMP 0x004`	`0xC004`	opcode `1100`, cond `000`, addr `000000100`
`BEQ 0x00C`	`0xD00C`	opcode `1101`, cond `000`, addr `000001100`
`BCC 0x011`	`0xD611`	opcode `1101`, cond `011`, addr `000010001`
`IN R1, 0x22`	`0xE222`	opcode `1110`, rd `001`, dir `0`, port `00100010`
`OUT 0x02, R3`	`0xE702`	opcode `1110`, rs `011`, dir `1`, port `00000010`
`HLT`	`0xF000`	opcode `1111`, remaining bits zero

Check your decoder against this table first

Every one of these encodings is produced by the supplied assembler and consumed correctly by the reference simulator. If your decoder disagrees with any row, fix that before running a single test program — a decoder bug looks like an execution bug and will waste hours.

3.6 Assembly language

The supplied assembler accepts one instruction per line, with optional labels and comments.

```
; comments start with a semicolon and run to end of line
LOOP:   LDI  R1, 0x0A      ; a label is a name followed by a colon
        ADD  R2, R2, R1    ; registers are R0..R7, case insensitive
        CMP  R2, R3
        BNE  LOOP          ; branch targets may be labels or numbers
        OUT  0x02, R2      ; port number first for OUT
        IN   R4, 0x00      ; register first for IN
        HLT
```

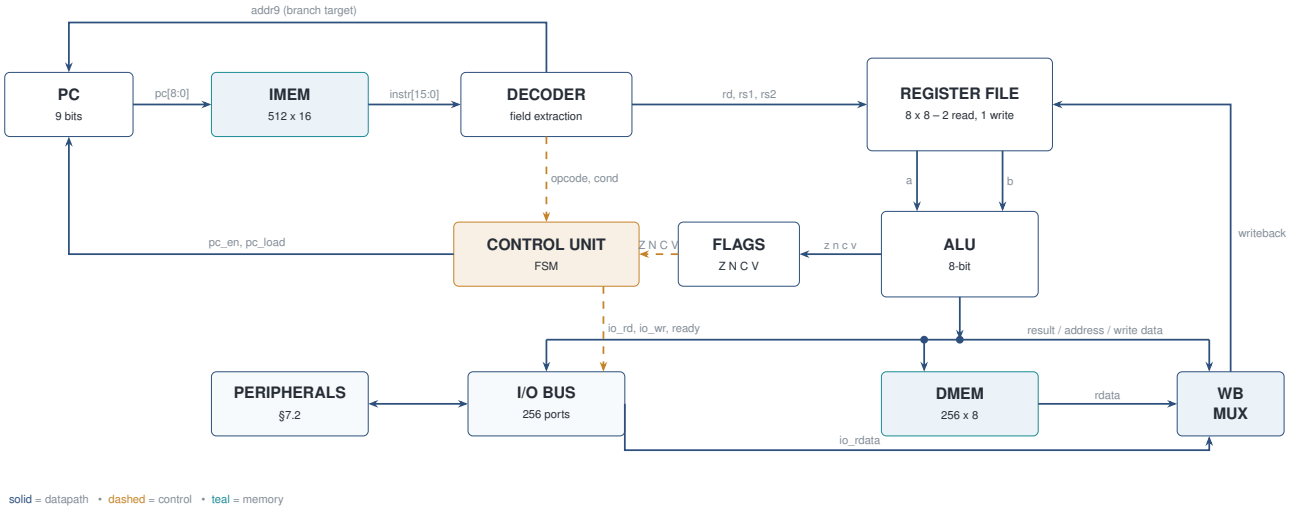
Two convenience mnemonics are provided. They are *not* additional opcodes; the assembler expands them into instructions from the table above.

You write	The assembler emits	Why
`MOV rd, rs`	`OR rd, rs, rs`	there is no dedicated register-to-register move
`NOT rd, rs`	`XOR rd, rs, rmask`	requires a register already holding `0xFF`

> **Things the instruction set deliberately does not have**
>
> No add-with-immediate, no register-to-register move, no subroutine call, no stack, no indexed addressing, no multiply. Each omission is a decision you will feel: to add a constant you must first `LDI` it into a register, which costs you an instruction and a register. Notice where this hurts as you write the test programs — *that* observation, written up in your report, is worth more than any amount of praise for the design.

4 System Architecture

4.1 Block diagram



How to read this diagram

The top row is the **fetch path**: the program counter addresses instruction memory, and the word that comes back is split into fields by the decoder. Everything below is the **execute path**: the register file supplies operands, the ALU computes, and the result goes back to a register, to data memory, or out to a port. The control unit sits in the middle and decides, cycle by cycle, which of those things happens.

4.2 Required module hierarchy

R12 Your design shall contain modules with *exactly* these names and *exactly* the port names and widths given in §5. The testbench harness binds to these names to extract the execution trace; a design that renames them will not run against the conformance suite.

Module	Owner	Responsibility
`dsd8_top`	shared	top level; instantiates everything below and exposes the board interface
`dsd8_cpu`	shared	the processor core: fetch, decode, execute, writeback
`prog_counter`	member A	the 9-bit PC and its next-address selection
`instr_memory`	member A	512 × 16 read-only memory, loaded by `\$readmemh`
`instr_decoder`	member A	splits the instruction word into fields; purely combinational
`register_file`	member B	8 × 8 registers, two read ports, one write port
`alu`	member B	arithmetic, logic, shifts, and the four flag outputs
`flags_reg`	member B	the architectural Z, N, C, V registers
`data_memory`	member C	256 × 8 read/write memory
`control_unit`	member C	the FSM that sequences every instruction
`io_bus`	member C	port address decode and the `io_ready` handshake

The ownership column is not a suggestion

Each member owns their modules and is individually examined on them at the oral defence. The interfaces between owners are in §5 and are frozen precisely so that three people can work in parallel without waiting for each other. Agree, in writing, who owns what at milestone M1.

4.3 Microarchitecture: your choice, with one constraint

R13 The microarchitecture is not specified. A single-cycle, multi-cycle or pipelined implementation are all acceptable, provided that instructions *retire in program order* and that the processor emits exactly one trace line per retired instruction (§8.2).

The recommended structure, and the one the control-unit table in §6 assumes, is a **multi-cycle FSM**. It is the easiest to get right, the easiest to debug, and it maps directly onto the control-unit material from the lectures.

Approach	Cycles / instr	Comment
Multi-cycle FSM	3 to 5	**Recommended.** Clear states, easy to trace, modest hardware.
Single-cycle	1	Simpler control, but one very long combinational path and a memory that must respond within the cycle.
Pipelined	1 (ideal)	Only after conformance is reached. Hazards must be handled, and retirement order must still be exact.

5 Module Interfaces

Every port list below is **frozen**. Copy them verbatim. All modules share the same clock and the same active-low synchronous reset.

5.1 `dsd8_top` — system top level

```
module dsd8_top (
  input  wire        clk,          // system clock
  input  wire        rst_n,        // synchronous, active low
  input  wire [7:0]  switches,     // board slide switches   -> port 0x00
  input  wire [4:0]  buttons,      // {btnU,btnD,btnL,btnR,btnC} -> port 0x01
  output wire [7:0]  leds,         // board LEDs             <- port 0x02
  output wire [6:0]  seg,          // seven-segment segments <- port 0x10
  output wire [3:0]  an,           // seven-segment digit enables
  output wire        halted        // asserted after HLT retires
);
```

5.2 `dsd8_cpu` — the processor core

```
module dsd8_cpu (
  input  wire        clk,
  input  wire        rst_n,
  // ---- instruction fetch interface ----
  output wire [8:0]  imem_addr,    // program counter
  input  wire [15:0] imem_data,    // instruction word, valid combinationally
  // ---- I/O bus (see section 6) ----
  output wire [7:0]  io_port,      // port number from the instruction
  output wire [7:0]  io_wdata,     // data for OUT
  input  wire [7:0]  io_rdata,     // data for IN
  output wire        io_rd,        // asserted for one access during IN
  output wire        io_wr,        // asserted for one access during OUT
  input  wire        io_ready,     // peripheral ready; low stalls the CPU
  // ---- status and trace ----
  output wire        halted,
  output wire        retire,       // one pulse per retired instruction
  output wire [8:0]  retire_pc,    // PC of the instruction just retired
  output wire [15:0] retire_ir,    // its instruction word
  output wire        retire_rf_we, // did it write a register?
  output wire [2:0]  retire_rf_addr,
  output wire [7:0]  retire_rf_data,
  output wire        retire_mem_we,// did it write data memory?
  output wire [7:0]  retire_mem_addr,
  output wire [7:0]  retire_mem_data,
  output wire [3:0]  retire_flags  // {Z,N,C,V} after the instruction
);
```

> **The `retire_*` outputs are the whole verification strategy**
>
> They are not debug decoration — they are how your processor is graded. The harness samples them on every cycle where `retire` is high and writes one trace line. Get these right early: a correct processor with a wrong trace scores zero on conformance. See §8.2.

5.3 `prog_counter`

```
module prog_counter (
  input  wire       clk,
  input  wire       rst_n,
  input  wire       pc_en,        // advance or load this cycle
  input  wire       pc_load,      // 1 = load branch_addr, 0 = increment
  input  wire [8:0] branch_addr,
  output reg  [8:0] pc
);
```

5.4 `instr_memory`

```
module instr_memory #(
  parameter IMAGE = "program.hex"   // hex image loaded with $readmemh
) (
  input  wire        clk,
  input  wire [8:0]  addr,
  output wire [15:0] data           // combinational read
);
```

5.5 `instr_decoder` — purely combinational

```
module instr_decoder (
  input  wire [15:0] instr,
  output wire [3:0]  opcode,       // instr[15:12]
  output wire [2:0]  rd,           // instr[11:9]
```

```
  output wire [2:0]  rs1,          // instr[8:6]
  output wire [2:0]  rs2,          // instr[5:3]
  output wire [7:0]  imm8,         // instr[7:0]
  output wire [8:0]  addr9,        // instr[8:0]
  output wire [2:0]  cond,         // instr[11:9]
  output wire [7:0]  port,         // instr[7:0]
  output wire        io_dir        // instr[8]: 0 = IN, 1 = OUT
);
```

> **Note the deliberate field overlaps**
>
> `rd` and `cond` are the same bits; `imm8` and `port` are the same bits; `rs1` and the top of `addr9` overlap. The decoder extracts all of them unconditionally and the control unit chooses which to believe, based on the opcode. This is normal, and it is why fixed-width instruction formats are worth having.

5.6 `register_file`

```
module register_file (
  input  wire       clk,
  input  wire       rst_n,
  input  wire [2:0] ra1,           // read address 1
  input  wire [2:0] ra2,           // read address 2
  output wire [7:0] rd1,           // combinational read data 1
  output wire [7:0] rd2,           // combinational read data 2
  input  wire       we,            // write enable
  input  wire [2:0] wa,            // write address
  input  wire [7:0] wd             // write data
);
```

R14 All eight registers shall be cleared to `0x00` by reset. Reads shall be combinational. A write and a read of the same register in the same cycle shall return the *old* value on the read ports; the new value becomes visible on the following cycle.

5.7 `alu`

```
module alu (
  input  wire [7:0] a,
  input  wire [7:0] b,
  input  wire [3:0] op,            // use the instruction opcode directly
  output wire [7:0] result,
  output wire       flag_z,
  output wire       flag_n,
  output wire       flag_c,
  output wire       flag_v
);
```

R15 The ALU shall be purely combinational and shall compute `ADD` and `SUB` with a single 9-bit adder, as specified in §2.3. Building two separate adders is permitted but must be justified at the design review.

5.8 `flags_reg`

```
module flags_reg (
  input  wire       clk,
  input  wire       rst_n,
  input  wire       flags_we,      // only for the eight flag-setting instructions
  input  wire [3:0] flags_in,      // {z,n,c,v} from the ALU
  output reg  [3:0] flags_out      // {Z,N,C,V} architectural state
);
```

5.9 `data_memory`

```
module data_memory (
  input  wire       clk,
  input  wire [7:0] addr,
  input  wire       we,
  input  wire [7:0] wdata,
  output wire [7:0] rdata          // combinational read
);
```

R16 All 256 bytes shall be cleared to `0x00` by reset, so that a read of a never-written location returns `0x00` rather than `x`.

5.10 `control_unit`

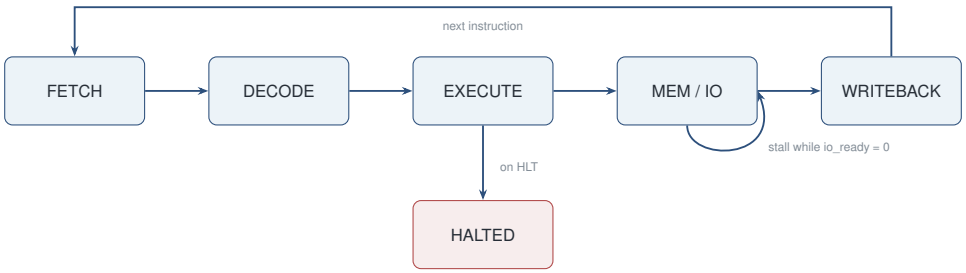
```
module control_unit (
  input  wire       clk,
  input  wire       rst_n,
```

```
  input  wire [3:0] opcode,
  input  wire [2:0] cond,
  input  wire [3:0] flags,         // {Z,N,C,V}
  input  wire       io_ready,
  // ---- datapath control ----
  output reg        pc_en,
  output reg        pc_load,
  output reg        rf_we,
  output reg        mem_we,
  output reg        io_rd,
  output reg        io_wr,
  output reg        flags_we,
  output reg  [1:0] wb_sel,        // 0=ALU 1=DMEM 2=imm8 3=io_rdata
  output reg        halted,
  output reg        retire         // one pulse as the instruction completes
);
```

R17 The control unit shall compute the branch decision from `cond` and `flags` exactly as tabulated in §3.3, and shall assert `pc_load` only when the branch is taken.

6 Control Unit and Datapath Operation

6.1 Recommended state machine



Instructions that need no memory or I/O access may skip states. A minimal implementation might use only `FETCH`, `EXECUTE` and `WRITEBACK`, taking three cycles for everything.

6.2 Control signal table

The table gives, for each opcode, the values the control unit must produce. `-` means “don’t care”.

Instr	rf_we	mem_we	io_rd	io_wr	flags_we	wb_sel	pc_load
`NOP`	0	0	0	0	0	–	0
`LDI`	1	0	0	0	0	2 (imm8)	0
`LD`	1	0	0	0	0	1 (DMEM)	0
`ST`	0	1	0	0	0	–	0
`ADD`	1	0	0	0	1	0 (ALU)	0
`SUB`	1	0	0	0	1	0	0
`AND`	1	0	0	0	1	0	0
`OR`	1	0	0	0	1	0	0
`XOR`	1	0	0	0	1	0	0
`SHL`	1	0	0	0	1	0	0
`SHR`	1	0	0	0	1	0	0
`CMP`	**0**	0	0	0	**1**	–	0
`JMP`	0	0	0	0	0	–	**1**
`B\emph{cc}`	0	0	0	0	0	–	**taken**
`IN`	1	0	1	0	0	3 (io)	0
`OUT`	0	0	0	1	0	–	0
`HLT`	0	0	0	0	0	–	0

> **The two rows that catch people**
>
> `CMP` is the only instruction that sets flags without writing a register — if `rf_we` is high for `CMP`, you will corrupt `R[rd]` with the subtraction result and the failure will look like a completely unrelated bug several instructions later. And `ST` writes memory but not a register; getting `rf_we` wrong there silently clobbers a register too.

6.3 Register operand routing

R18 The address applied to register read port 1 shall be `rs1` for all register-operand instructions. For `ST`, read port 2 shall be addressed by `rd` — because in Format M the *stored value* comes from the `rd` field while the *address* comes from `rs1`.

This asymmetry is the most common wiring mistake in the whole project. Write it on a card.

Instruction	Read port 1 (`ra1`)	Read port 2 (`ra2`)	Write address
R-type	`rs1`	`rs2`	`rd`
`LD`	`rs1` (address)	–	`rd`
`ST`	`rs1` (address)	`rd` (data)	none
`LDI`	–	–	`rd`
`IN`	–	–	`rd`
`OUT`	–	`rd` (data)	none

7 Input / Output Subsystem

7.1 Bus protocol

The I/O bus is a simple synchronous request/acknowledge interface. The processor presents a port number and a direction; the addressed peripheral responds, and may take as long as it needs.

Signal	Direction	Width	Meaning
`io_port`	CPU → bus	8	port number, from `IR[7:0]`
`io_wdata`	CPU → bus	8	data to be written by `OUT`
`io_rdata`	bus → CPU	8	data returned to `IN`
`io_rd`	CPU → bus	1	read request, held until `io_ready`
`io_wr`	CPU → bus	1	write request, held until `io_ready`
`io_ready`	bus → CPU	1	peripheral is ready to complete the transfer

R19 The processor shall assert `io_rd` or `io_wr` and hold `io_port` and `io_wdata` stable until it observes `io_ready` high on a rising clock edge. It shall then deassert the request and complete the instruction. While waiting it shall change no architectural state.

R20 A peripheral shall not require the request to be deasserted between transfers, and shall not assert `io_ready` unless a request is active.

> **Why a ready signal, when nothing needs it yet?**
> Because it costs almost nothing now and it is the difference between a control unit that can wait and one that cannot. Test group T5 attaches a deliberately slow peripheral that holds `io_ready` low for a pseudo-random number of cycles. A processor that ignores the handshake passes every other test and fails that one — which is exactly the point.

7.2 Standard port map

R21 Ports `0x00` to `0x3F` are reserved and shall behave exactly as tabulated. Reads from any unmapped port shall return `0x00`; writes to any unmapped port shall be discarded. No port shall ever hold `io_ready` low indefinitely.

Port	Name	Access	Function
0x00	SWITCHES	read	the eight slide switches
0x01	BUTTONS	read	{3'b000, btnU, btnD, btnL, btnR, btnC}, already debounced
0x02	LEDS	write	the eight LEDs; the value is latched
0x10	SEG_VALUE	write	value shown on the seven-segment display, in hexadecimal
0x11	SEG_ENABLE	write	bit n enables digit n; 0x00 blanks the display
0x20	UART_TX	write	byte to transmit; ignored while the transmitter is busy
0x21	UART_RX	read	the received byte; reading it clears rx_valid
0x22	UART_STATUS	read	bit 0 = tx_busy, bit 1 = rx_valid, others zero
0x30	TIMER_COUNT	read	low byte of a free-running counter
0x31	TIMER_CTRL	write	bit 0 = enable, bit 1 = clear
0x40	SLOW_ECHO	r/w	**test peripheral**: returns the last byte written, after a delay of 0 to 7 cycles during which it holds io_ready low

Reference modules are supplied for the UART, the seven-segment driver, the timer and SLOW_ECHO

You may use them as given, or replace them with your own. They are provided so that a fault in a peripheral cannot masquerade as a fault in your processor: if a conformance test fails with the supplied peripherals in place, the problem is in your CPU.

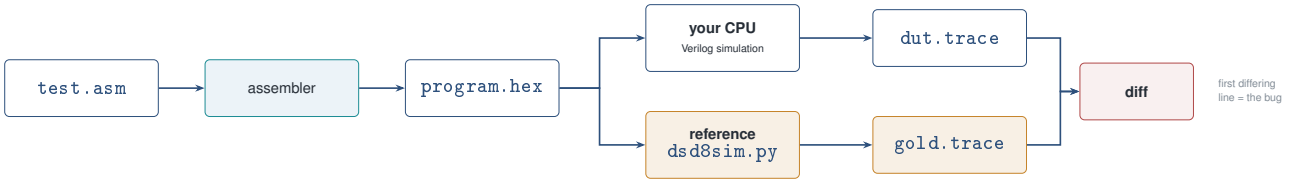
7.3 Polling example

```
; wait for a character, then echo it back
WAIT:   IN   R1, 0x22        ; read UART status
        LDI  R2, 0x02        ; rx_valid is bit 1
        AND  R3, R1, R2      ; sets Z if the bit is clear
        BEQ  WAIT            ; nothing yet, keep polling
        IN   R4, 0x21        ; take the byte
        OUT  0x20, R4        ; send it straight back
        JMP  WAIT
```

8 Verification

8.1 The strategy, in one paragraph

A processor has more reachable states than you can ever enumerate, so it cannot be verified by inspecting waveforms. Instead you run the same program on your processor and on a **golden reference simulator**, have both emit one line of text per instruction, and compare the two files. When they diverge, the *first* differing line tells you which instruction broke and what it did wrong. This is how real processors are verified, and it is the single most transferable skill in this project.



8.2 Execution trace format

R22 The processor shall emit exactly **one trace line per retired instruction**, in program order. No line shall be emitted for a cycle in which no instruction retires, and no instruction shall produce two lines. The line for the `HLT` instruction is emitted; nothing is emitted thereafter.

Each line contains ten space-separated fields:

Field	Name	Format	Contents
1	sequence	`%04d`	instruction count, starting at `0001`
2	pc	`%03X`	address of the instruction that just retired
3	ir	`%04X`	the instruction word
4	rf_we	`%01d`	1 if a register was written, else 0
5	rf_addr	`%01d`	which register, else 0
6	rf_data	`%02X`	the value written, else `00`
7	mem_we	`%01d`	1 if data memory was written, else 0
8	mem_addr	`%02X`	the address written, else `00`
9	mem_data	`%02X`	the byte written, else `00`
10	flags	`%01d%01d%01d%01d`	Z, N, C, V *after* the instruction completed

A real extract, from the multiply example of §B:

```
0001 000 140D 1 1 0D 0 00 00 0000     ; LDI R1, 13
0002 001 1607 1 2 07 0 00 00 0000     ; LDI R2, 7
0003 002 1800 1 3 00 0 00 00 0000     ; LDI R3, 0
0004 003 1A01 1 4 01 0 00 00 0000     ; LDI R4, 1
```

R23 Fields 5, 6, 8 and 9 shall be zero whenever the corresponding write-enable is zero. A processor that leaves stale data in these fields will fail the comparison even though its architectural behaviour is correct.

> **The most common trace bug, and how to recognise it**
>
> If your trace is correct except that every line is shifted by one instruction, you are sampling the `retire\_*` outputs one cycle too late or too early. If the flags field is right for the *previous* instruction, you are sampling flags before the flag register updates. Both look catastrophic in the diff and are one-line fixes.

8.3 Harness

The supplied harness (`tb_dsd8.v`) instantiates your `dsd8_cpu`, loads a hex image, runs until `halted` or a cycle limit, and writes the trace. You do not modify it.

```
always @(posedge clk) begin
  if (rst_n && retire) begin
    $fdisplay(f, "%04d %03X %04X %01d %01d %02X %01d %02X %02X %01d%01d%01d%01d",
              seq, retire_pc, retire_ir,
              retire_rf_we,  retire_rf_addr,  retire_rf_data,
              retire_mem_we, retire_mem_addr, retire_mem_data,
              retire_flags[3], retire_flags[2], retire_flags[1], retire_flags[0]);
    seq <= seq + 1;
  end
end
```

8.4 Conformance test suite

Forty programs in six groups. Each is a numbered assembly file with a stored golden trace. Your score for Part A is simply how many of them your processor reproduces exactly.

Group T1 — data movement and immediates (T01–T08)

Test	What it checks
T01	`LDI` into each of the eight registers in turn; every register must be independently writable
T02	`LDI` with the boundary values `0x00`, `0x01`, `0x7F`, `0x80`, `0xFF`
T03	`MOV` (assembled as `OR rd,rs,rs`) between every pair of registers
T04	`LDI` does not disturb the flags: set flags, then `LDI`, then branch on the old flags
T05	`NOP` changes nothing at all
T06	writing a register then immediately reading it as a source operand
T07	reading the same register on both ALU input ports (`ADD R1,R2,R2`)
T08	`rd` equal to `rs1` equal to `rs2` (`ADD R3,R3,R3`)

Group T2 — data memory (T09–T14)

Test	What it checks
T09	reading never-written locations: every one must return `0x00` (see §2.5)
T10	store then load from the same address; store to `0x00` and to `0xFF`
T11	the address register is the *contents* of `rs1`, not the register number
T12	`ST [Rx], Rx` — storing a register using itself as the address
T13	a walking-ones pattern written across all 256 locations and read back
T14	`LD` does not disturb the flags

Group T3 — ALU results and flags (T15–T22)

Test	What it checks
T15	`ADD` across the carry boundary: `0xFF+0x01`, `0x80+0x80`, `0x7F+0x01`
T16	`SUB` and the borrow convention: `C=1` must mean *no* borrow
T17	**signed overflow**: `0x7F+0x01` sets V; `0xFF+0x01` does not
T18	`AND`, `OR`, `XOR` clear C and V and set Z and N from the result
T19	`SHL` out of bit 7 into carry, and `SHR` out of bit 0 into carry
T20	`SHR` is *logical*: shifting `0x80` gives `0x40`, never `0xC0`
T21	`CMP` sets flags and writes **no** register
T22	an arithmetic right shift constructed from the available instructions

Group T4 — control flow (T23–T30)

Test	What it checks
T23	`JMP` forwards, backwards, to address `0x000` and to the last used address
T24	each of the eight branch conditions taken
T25	each of the eight branch conditions *not* taken — execution must fall through
T26	a branch to itself, executed once then escaped
T27	branches do not modify flags: branch, then branch again on the same condition
T28	unsigned ordering with `BCS`/`BCC` across `0x7F`/`0x80`
T29	signed ordering built from `N` and `V`
T30	a loop that executes exactly 256 times and terminates

Group T5 — input and output (T31–T35)

Test	What it checks
T31	`OUT` to `LEDS` then `IN` from `SWITCHES`; the two spaces are independent
T32	`IN` from an unmapped port returns `0x00`; `OUT` to one is harmless
T33	I/O does not disturb data memory, and data memory does not disturb I/O
T34	**the `io\_ready` handshake**: repeated access to `SLOW_ECHO`, which stalls 0–7 cycles at random. The CPU must wait and must not lose or duplicate a transfer.
T35	`IN` and `OUT` leave the flags unchanged

Group T6 — integration programs (T36–T40)

Test	What it checks
T36	8-bit multiply by repeated addition
T37	population count using `SHR` and the carry flag
T38	bubble sort of eight bytes held in data memory
T39	find the maximum and minimum of a block of memory
T40	polled UART echo, driven by a scripted stimulus

Recommended order of attack

Do not try to pass tests in numerical order. Get **T05** (`NOP`) passing first — it proves fetch, the PC, and the trace all work. Then **T01**, which proves register writeback. Then **T23** (`JMP`), which proves the branch path. With those three, the machine runs; everything after is arithmetic detail. Teams that begin with `ADD` typically spend two days not knowing whether the bug is in the ALU, the decoder or the trace.

8.5 Your own verification — required beyond conformance

Passing the conformance suite proves your processor works on forty programs. It does not prove the modules are right, and it will not help you when test T36 fails for a reason buried in the ALU.

R24 Each team shall additionally deliver: a **self-checking unit testbench for every module** in §5; a **constrained-random ALU test** comparing at least 10 000 operand pairs against a behavioural reference, covering all eight ALU operations; and a **written verification report** stating what has been verified, what has not, and why the remaining risk is acceptable.

R25 The ALU test shall demonstrate that the corner cases `0x00`, `0x01`, `0x7F`, `0x80` and `0xFF` occur as both operands, and shall report this as coverage evidence rather than asserting it.

Bug log

Keep a running log of every bug found: what it was, how it was found (unit test, conformance diff, review, inspection), and how long it took. It is a graded artifact, and at the end of the semester the class data is aggregated. The pattern that emerges every year — that bugs found by unit tests cost minutes and bugs found at integration cost days — is more persuasive from your own numbers than from any lecture.

9 Deliverables, Milestones and Assessment

9.1 Milestones

ID	Due	Deliverable	Weight
M1	End W11	Team formed and module ownership agreed in writing. Block diagram annotated with your chosen microarchitecture. Control-signal table completed for all sixteen opcodes. Repository created with the directory structure and a CI script that runs an empty regression. *No RTL is graded.*	6 %
M2	End W12	`alu`, `register_file` and `instr_decoder` complete, each with a self-checking unit testbench. The ALU random test running.	6 %
M3	End W13	Core integrated and fetching. Tests T05, T01 and T23 passing. Trace output in the correct format.	6 %
M4	End W14	Groups T1–T4 passing (30 of 40 tests). Bug log populated.	6 %
M5	End W15	All 40 conformance tests passing. Verification report drafted. Design review with a partner team completed and findings answered in writing.	6 %
M6	W16	Final repository, report, demonstration and individual oral defence.	10 %

9.2 Mark allocation

Component	Weight	How it is judged
Conformance	30 %	Objectively: the fraction of the 40 tests reproduced exactly. No partial credit per test — a trace either matches or it does not.
Milestones M1–M5	20 %	On time and complete, as tabulated above.
Verification quality	20 %	Unit testbenches, random ALU test, coverage evidence, the closure argument in the verification report, and the bug log.
Design quality and documentation	15 %	Lint-clean RTL; no inferred latches; readable hierarchy; the design-decision record; the final report.
Individual oral defence	15 %	Each member is examined separately on the modules they own and on the system as a whole.

The defence is a multiplier, not an addition

Your share of the team marks is scaled by your individual oral defence: 1.0 for an authoritative performance, down to 0.4 for a student who cannot trace a signal through their own module. This is announced now, in week 11, precisely so that nobody is surprised in week 16.

9.3 Final deliverables

1. **Repository** with `rtl/`, `tb/`, `sim/`, `doc/` and `scripts/`, and a CI script that runs every unit test and the full conformance suite and returns a non-zero exit code on any failure.
2. **Conformance report** generated by the supplied script: which tests pass, which fail, and for each failure the first differing trace line.
3. **Verification report**, 4–6 pages: test plan, what each unit testbench covers, ALU coverage evidence, the closure argument, and the bug log with the *found by* field completed.
4. **Design report**, 6–10 pages: microarchitecture description, the state machine, the control-signal table as implemented, and a **design-decision record** of at least five decisions with the alternatives you rejected and why.
5. **Demonstration**: your processor running at least two of the T6 integration programs, with the result visible on LEDs, the seven-segment display or a terminal.
6. **Individual oral defence**, 10 minutes per member, no notes.

9.4 Sample defence questions

These are published so that you prepare by understanding rather than by memorising.

- Point to the logic that produces `flags\_we`. Walk me from the instruction word to that signal.
- What happens if `io\_ready` goes low for two hundred cycles? Show me where your design waits.
- Why does `CMP` not write a register? What did you have to do differently to achieve that?
- Your trace showed the right value one instruction late. How did you find that, and what was it?
- I want to add an `ADDI` instruction. Do not write code — tell me every module you would touch and what would break.
- Show me the coverage bin that proves you tested signed overflow, and tell me what it does *not* prove.

Appendix A — Encoding Quick Reference

Print this page and keep it beside you. Bit 15 is on the left.

Op	Assembly	Fmt	Bit layout, 15 down to 0
0	NOP	—	0000 000 0 00000000
1	LDI rd,imm	I	0001 ddd 0 iiiiiiii
2	LD rd,[rs1]	M	0010 ddd aaa 000000
3	ST [rs1],rs2	M	0011 sss aaa 000000 (*sss = data, aaa = address*)
4	ADD rd,rs1,rs2	R	0100 ddd aaa bbb 000
5	SUB rd,rs1,rs2	R	0101 ddd aaa bbb 000
6	AND rd,rs1,rs2	R	0110 ddd aaa bbb 000
7	OR rd,rs1,rs2	R	0111 ddd aaa bbb 000
8	XOR rd,rs1,rs2	R	1000 ddd aaa bbb 000
9	SHL rd,rs1	R	1001 ddd aaa 000 000
A	SHR rd,rs1	R	1010 ddd aaa 000 000
B	CMP rs1,rs2	R	1011 000 aaa bbb 000
C	JMP addr	J	1100 000 tttttttt t
D	Bcc addr	J	1101 ccc tttttttt t
E	IN rd,port	P	1110 ddd 0 pppppppp
E	OUT port,rs	P	1110 sss 1 pppppppp
F	HLT	—	1111 000 0 00000000

cond	Mnemonic	Taken if	Flag effects, at a glance
000	BEQ	Z=1	ADD SUB CMP set all four flags. AND OR XOR set Z and N, clear C and V.
001	BNE	Z=0	
010	BCS	C=1	
011	BCC	C=0	
100	BMI	N=1	
101	BPL	N=0	
110	BVS	V=1	
111	BVC	V=0	

Appendix B — Worked Example Programs

Every program below has been assembled and executed on the reference simulator; the results quoted are the actual outputs. Use them to check your processor as soon as it runs.

B.1 Multiply by repeated addition

Multiplies `R1` by `R2`, leaving the product in `R3` and writing it to the LEDs.

```
        LDI  R1, 13         ; multiplicand
        LDI  R2, 7          ; multiplier  (counts down to zero)
        LDI  R3, 0          ; running product
        LDI  R4, 1          ; the constant 1 -- there is no "subtract immediate"
LOOP:   LDI  R5, 0
        CMP  R2, R5         ; has the multiplier reached zero?
        BEQ  DONE
        ADD  R3, R3, R1     ; product += multiplicand
        SUB  R2, R2, R4     ; multiplier -= 1
        JMP  LOOP
DONE:   OUT  0x02, R3       ; show the product on the LEDs
        HLT
```

Property	Value
Hex image	`120D 1407 1600 1801 1A00 B0A8 D00A 46C8 54A0 C004 E702 F000`
Labels	`LOOP = 0x004`, `DONE = 0x00A`
Result	`R3 = 91` (that is 13×7), LEDs `0x5B`
Instructions retired	51

The first six lines of the reference trace, which your processor must reproduce exactly:

```
0001 000 120D 1 1 0D 0 00 00 0000
0002 001 1407 1 2 07 0 00 00 0000
0003 002 1600 1 3 00 0 00 00 0000
0004 003 1801 1 4 01 0 00 00 0000
0005 004 1A00 1 5 00 0 00 00 0000
0006 005 B0A8 0 0 00 0 00 00 0010
```

Read line 6 carefully — it is the best single self-check in this document

The instruction is `CMP R2, R5`, comparing 7 against 0. Fields 4 to 9 are all zero because `CMP` writes **no** register and **no** memory. The flags are `0010`: Z=0 ($7 \neq 0$), N=0 (the result 7 is positive), **C=1** ($7 \geq 0$, so no borrow), V=0. If your processor writes a register here, or reports C=0, you have found two of the three classic DSD-8 bugs before writing a test.

B.2 Population count

Counts the number of set bits in `R1` by shifting them one at a time into the carry flag.

```
        LDI  R1, 0xB7       ; 1011 0111 -- six bits set
        LDI  R2, 0          ; count
        LDI  R3, 8          ; bits remaining
        LDI  R4, 1
LOOP:   SHR  R1, R1         ; the old bit 0 lands in C
        BCC  SKIP           ; C = 0, so that bit was clear
        ADD  R2, R2, R4
SKIP:   SUB  R3, R3, R4
        BNE  LOOP           ; SUB set Z when the counter hit zero
        OUT  0x02, R2
        HLT
```

Result: `R2 = 6`, 44 instructions retired. Note how `SUB` sets `Z` and the very next instruction branches on it — that is the intended idiom, and it works only because `BNE` does not itself disturb the flags.

B.3 Bubble sort in data memory

Sorts eight bytes held at `DMEM[0x00]` through `DMEM[0x07]` into ascending order.

```
        LDI  R1, 8          ; n
        LDI  R7, 1          ; the constant 1
```

```
OUTER:  LDI  R2, 0          ; i = 0
        LDI  R6, 0          ; "a swap happened" flag
INNER:  MOV  R3, R2         ; assembles to OR R3, R2, R2
        ADD  R4, R3, R7     ; R4 = i + 1
        CMP  R4, R1
        BCS  ENDIN          ; i+1 >= n, so this pass is finished
        LD   R5, [R3]       ; a[i]
        LD   R0, [R4]       ; a[i+1]
        CMP  R5, R0
        BCC  NOSWAP         ; a[i] < a[i+1], already in order
        BEQ  NOSWAP         ; equal, nothing to do
        ST   [R3], R0       ; swap them
        ST   [R4], R5
        LDI  R6, 1
NOSWAP: ADD  R2, R2, R7
        JMP  INNER
ENDIN:  LDI  R0, 0
        CMP  R6, R0
        BNE  OUTER          ; repeat while any swap occurred
        HLT
```

Property	Value
Input	`DMEM[0..7] = 5 3 9 1 7 2 8 4`
Output	`DMEM[0..7] = 1 2 3 4 5 7 8 9`
Instructions retired	454

Notice what this program cost

Twenty-two instructions, and four of them exist only because the instruction set has no add-immediate and no register-to-register move. `R7` is permanently occupied holding the constant 1. This is exactly the kind of observation your design report should contain: not “the ISA is limited”, but “here is the program, here is where the limitation cost me, and here is the one instruction I would add.”

B.4 Polled UART echo

```
WAIT:   IN   R1, 0x22       ; UART status register
        LDI  R2, 0x02       ; rx_valid is bit 1
        AND  R3, R1, R2     ; Z is set if the bit is clear
        BEQ  WAIT
        IN   R4, 0x21       ; take the received byte
        OUT  0x20, R4       ; echo it straight back
        JMP  WAIT
```

With the characters `H`, `i`, `!` presented by the test stimulus, the transmitted sequence is `Hi!`. This program never halts, so the harness stops it at the cycle limit — which is the expected behaviour for test T40.

Appendix C — Common Pitfalls

Collected from the failure modes this design reliably produces. Each entry names the symptom first, because that is what you will actually see.

Symptom	Almost always the cause
Every trace line is right but shifted by one instruction	The `retire\_*` outputs are sampled a cycle early or late. Check that `retire` pulses in the same cycle the architectural state actually updates.
The flags field lags one instruction behind	You are reading the flag register before it updates. The trace must show the flags *after* the instruction completes.
`BCS`/`BCC` branch the wrong way after `CMP`	The carry convention. In DSD-8, `C=1` means *no borrow*, so `C=1` means $\geq$. See §2.3.
A register is corrupted several instructions after a comparison	`rf_we` is asserted for `CMP`. `CMP` sets flags and writes nothing.
`ST` writes the right byte to the wrong address, or the wrong byte to the right address	The Format M operand routing. The address comes from `rs1`; the stored data comes from `rd`. See §6.3.
Everything works until test T34	The `io_ready` handshake is being ignored. Your CPU completes the transfer regardless of whether the peripheral was ready.
Reads of data memory return `x`	Data memory was not cleared on reset. Requirements R6 and R16.
The first instruction executes twice, or is skipped	The PC increment and the reset release overlap. Check what `pc_en` does during the cycle in which reset deasserts.
Signed comparisons behave oddly near `0x80`	You used `BMI` where the correct test is `N` $\neq$ `V`. `BMI` alone is only correct when no overflow occurred.
`SHR` of `0x80` gives `0xC0`	You implemented an arithmetic shift. DSD-8's `SHR` is logical and always shifts in a zero.
The design simulates but will not synthesise	Almost always an inferred latch, from a `case` without a `default` or an `always @(*)` block that does not assign its outputs on every path.

A closing note on method

The processor is not the hard part. Sixteen opcodes and eight registers is a small machine, and a careful team will have it fetching within a week. What separates the projects at the end of the semester is whether you can *show* that it is right — and whether, when it is not, you can find out why from evidence instead of by staring at the code.

Build the trace output before you build the ALU. Get `NOP` passing before you get `ADD` passing. Write the unit testbench before the module it tests. Keep the bug log honestly. Those four habits are worth more than any amount of cleverness in the datapath, and they are the ones you will still be using in ten years.